

● 阶段二：进阶期 - RAG 应用

📖 本文档为《AI 前端开发体系化学习指南》的阶段拆分文档
完整指南请查看：[学习指南总览](#)

🎯 **阶段目标：**打破 LLM 知识截止限制，构建基于私有数据的智能问答系统。

📑 本章目录

- [核心能力指标](#)
- [核心概念解析](#)
 - [RAG 架构全景图](#)
 - [核心概念对照表](#)
 - [RAG 深入原理](#)
 - [Lost in the Middle 问题](#)
 - [高级 RAG 范式](#)
- [环境搭建](#)
- [核心实现](#)
 - [文档加载与解析](#)
 - [智能分块策略](#)
 - [向量化与存储](#)
 - [RAG 链构建](#)
- [实战项目](#)

💡 你将学到

- RAG（检索增强生成）的完整架构与工作原理
- 文本分块（Chunking）策略与 Embedding 向量化技术
- 使用 [LangChain.js](#) 搭建企业级 RAG 流水线
- 向量数据库（[Pinecone](#)）的集成与语义搜索
- 检索质量优化：重排序、混合检索、查询扩展

🔗 前置知识

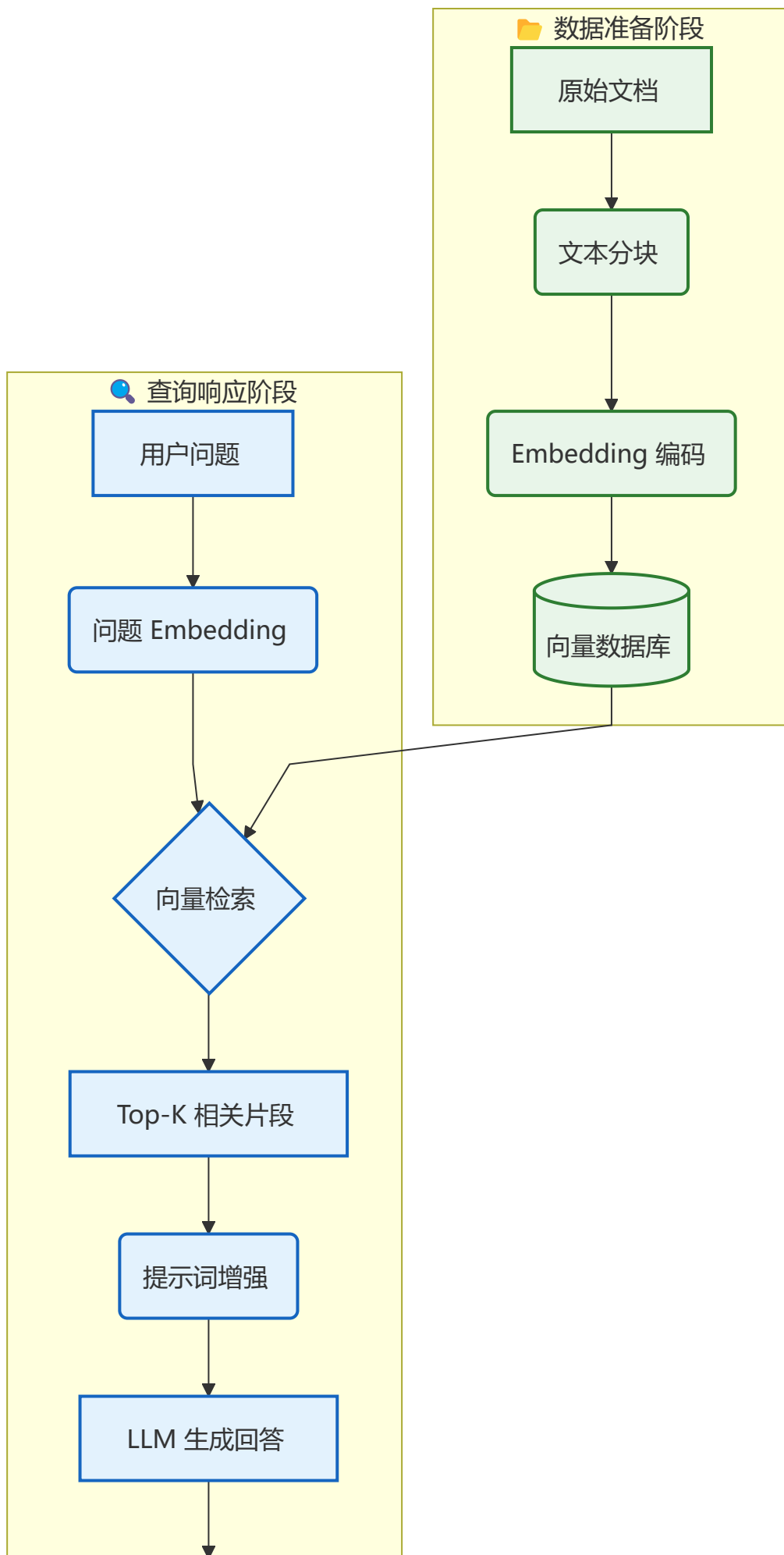
- 完成 [● 阶段一：入门期](#)
- 熟悉 Node.js 文件系统与异步编程
- 了解 HTTP API 与基础网络概念

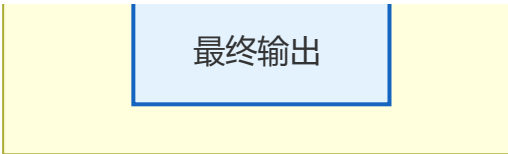
核心能力指标

- ☐ 深入理解 RAG (检索增强生成) 架构原理
- ☐ 掌握文本分块 (Chunking) 与向量化 (Embedding) 技术
- ☐ 熟练使用向量数据库 ([Pinecone](#)/[Milvus](#)/[Chroma](#)) 进行语义搜索
- ☐ 构建完整的 检索 -> 增强 -> 生成 流水线
- ☐ 掌握检索质量优化策略 (重排序、混合检索)

核心概念解析

2.1 RAG 架构全景图





2.2 核心概念对照表

概念	通俗解释	技术实现
Embedding	将文字翻译成数学坐标	<code>text-embedding-3-small</code> (1536 维)
向量数据库	语义搜索引擎	Pinecone, Milvus, Chroma
相似度	坐标轴上的距离	余弦相似度 (Cosine Similarity)
分块 (Chunking)	把长文章切成小段落	固定长度、递归字符、语义分块
上下文窗口	LLM 的短期记忆容量	4K, 8K, 32K, 128K Tokens

2.3 RAG 深入原理

RAG vs 微调的本质区别：

维度	RAG	微调 (Fine-tuning)
知识来源	外部知识库，动态更新	模型内部参数，静态
更新成本	低——只需更新知识库重建索引	高——需要重新训练模型
幻觉控制	✅ 引用来源，可验证	❌ 依赖记忆，易幻觉
长尾知识	✅ 检索即可获得	❌ 难以记住
适用场景	知识密集型问答	格式/风格/行为对齐

什么时候用RAG，什么时候用微调：

- 客服问答（知识更新频繁） → **RAG**
- 知识库查询、需引用来源 → **RAG**
- 模型行为/回答风格定制 → **微调**
- 两者结合最佳：微调让模型学会"回答格式"，RAG提供实时知识

Chunking（文本切块）是影响检索质量的最关键因素之一：

切块大小	优点	缺点	适用场景
小 (256 tokens)	精度高，噪声少	可能切碎语义	QA精确匹配
中 (512 tokens)	平衡	平衡	通用推荐
大 (1024 tokens)	语义完整	含噪声	摘要类任务

相邻块保留10-20%重叠，防止关键信息被切在边界上。先进的语义切块（Semantic Chunking）在句子边界根据语义变化动态切分。

2.4 Lost in the Middle 问题

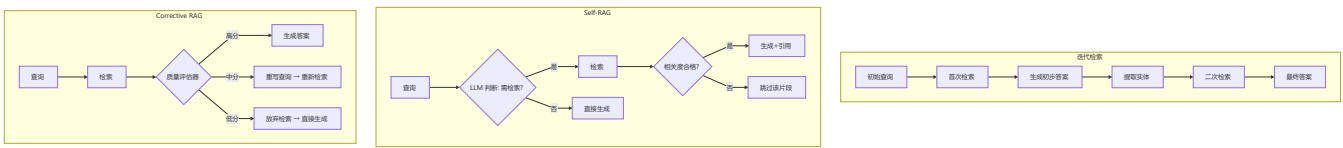
LLM对长上下文中"中间位置"信息的利用率显著低于开头和结尾

现象：检索到的10个文档中，放在开头和结尾的文档利用率高，中间的容易被"忽略"。如果最相关的文档恰好在中间，检索质量会大幅下降。

缓解方法：

- "重要放两端"：检索结果按相关度排序，最相关放最前或最后
- 减少Top-K：从10降到3-5，减少中间位置
- 重排序强调：用Re-ranker确保高质量内容排在两端

2.5 高级RAG范式



范式	原理	适用场景	实现要点
迭代检索	先生成初步答案，提取关键实体再次检索	复杂事实性问答	实体提取依赖 NER 模型或 LLM 自身
自适应检索 (Self-RAG)	LLM 自行判断是否需要检索、检索什么	混合型问答	需在 Prompt 中植入反思 Token，训练专门的判断头
Corrective RAG	检索后评估质量，差则重写查询重新检索	检索质量不稳定场景	需要训练/配置独立的相关度分类器（如 bge-reranker）
Agentic RAG	用 Agent 编排多步检索+工具调用+推理	复杂任务规划	引入 ReAct 循环，管理 Tool Registry

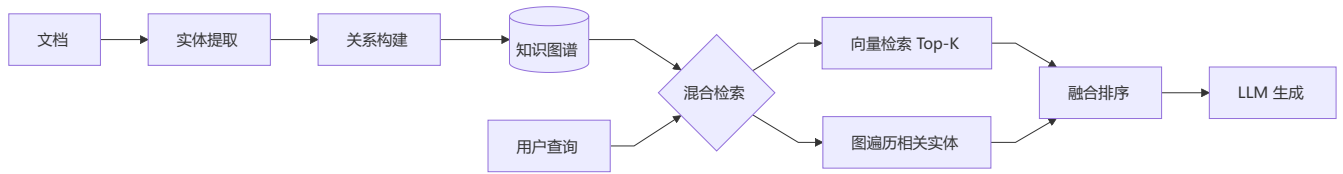
```
// Corrective RAG 核心实现
async function correctiveRAG(query: string) {
  const docs = await retrieve(query);
  const relevanceScore = await evaluateRelevance(query, docs);

  if (relevanceScore > 0.7) {
    return generate(query, docs); // ✓ 高质量 → 直接生成
  } else if (relevanceScore > 0.3) {
    const rewritten = await rewriteQuery(query, docs); // ✎ 中等 → 重写查询
    const newDocs = await retrieve(rewritten);
    return generate(query, newDocs);
  } else {
    return generate(query); // ✗ 低质量 → 放弃检索
  }
}
```

```
async function evaluateRelevance(query: string, docs: string[]): Promise<number> {
  const scores = await Promise.all(docs.map(d => crossEncoder(query, d)));
  return Math.max(...scores);
}
```

2.6 GraphRAG —— 知识图谱增强检索

传统 RAG 以文档块为检索单元，GraphRAG 引入知识图谱来捕捉实体间关系，擅长多跳推理。



特性	传统 RAG	GraphRAG
检索粒度	文本块 (Chunk)	实体 + 关系三元组
多跳推理	弱 (依赖文本覆盖)	强 (沿关系路径遍历)
实现复杂度	低	高 (需实体抽取 + 图存储)
适用场景	事实问答、文档摘要	复杂关系推理、报告生成

```
// GraphRAG 检索实现
interface Entity { name: string; type: string; embedding: number[] }
interface Relation { source: Entity; target: Entity; label: string }

class GraphRAG {
  async retrieve(query: string): Promise<string[]> {
    const queryEmb = await embed(query);
    // ① 向量检索实体
    const entities = await vectorSearch(queryEmb, { collection: 'entities', topK: 5 });
    // ② 图遍历: 从命中实体出发, 沿关系扩展到 2 跳
    const subgraph = await this.traverse(entities.map(e => e.name), 2);
    // ③ 将子图序列化为文本上下文
    return subgraph.map(e => `${e.name} --[${e.label}]--> ${e.target}`);
  }
}
```

🔧 环境搭建

2.3 项目初始化

```
# 🚀 创建项目
npx create-next-app@latest rag-app --typescript --tailwind --app

cd rag-app
```

```
# 📦 安装 LangChain.js 生态
npm install langchain @langchain/openai @langchain/pinecone
npm install @pinecone-database/pinecone

# 📄 安装文档解析依赖
npm install pdf-parse mammoth cheerio
```

2.4 环境变量配置

```
# .env.local
OPENAI_API_KEY=sk-your-openai-key
PINECONE_API_KEY=pc-your-pinecone-key
PINECONE_ENVIRONMENT=us-east-1
PINECONE_INDEX_NAME=rag-documents
```

核心实现

2.5 文档加载与解析

```
// lib/document-loaders.ts
import * as fs from 'fs/promises';
import pdfParse from 'pdf-parse';
import mammoth from 'mammoth';

export interface Document {
  content: string;
  metadata: { source: string; type: string; [key: string]: unknown };
}

export class DocumentLoader {
  static async loadPDF(filePath: string): Promise<Document> {
    const buffer = await fs.readFile(filePath);
    const result = await pdfParse(buffer);
    return {
      content: result.text,
      metadata: { source: filePath, type: 'pdf', pages: result.numpages },
    };
  }

  static async loadWord(filePath: string): Promise<Document> {
    const buffer = await fs.readFile(filePath);
    const result = await mammoth.extractRawText({ buffer });
    return { content: result.value, metadata: { source: filePath, type: 'docx' } };
  }

  static async loadText(filePath: string): Promise<Document> {
    const content = await fs.readFile(filePath, 'utf-8');
    return { content, metadata: { source: filePath, type: 'txt' } };
  }
}
```

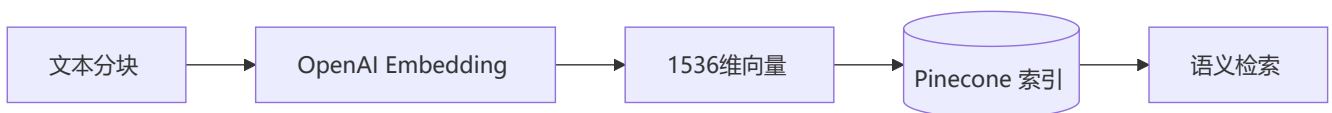
2.6 智能分块策略

```
// lib/text-splitter.ts
export class TextSplitter {
  // 📄 递归字符分块 (推荐)
  static splitRecursive(text: string, chunkSize = 1000, overlap = 200): string[] {
    const chunks: string[] = [];
    let start = 0;
    while (start < text.length) {
      const end = Math.min(start + chunkSize, text.length);
      chunks.push(text.slice(start, end));
      if (end >= text.length) break;
      start = end - overlap;
    }
    return chunks;
  }

  // 📄 按段落分块 (保持语义完整性)
  static splitByParagraphs(text: string, chunkSize = 1000): string[] {
    const paragraphs = text.split(/\n\s*\n/);
    const chunks: string[] = [];
    let currentChunk = '';

    for (const para of paragraphs) {
      if ((currentChunk + para).length > chunkSize && currentChunk) {
        chunks.push(currentChunk.trim());
        currentChunk = para;
      } else {
        currentChunk = currentChunk ? currentChunk + '\n\n' + para : para;
      }
    }
    if (currentChunk) chunks.push(currentChunk.trim());
    return chunks;
  }
}
```

2.7 向量化与存储



```
// lib/vector-store.ts
import { OpenAIEmbeddings } from '@langchain/openai';
import { Pinecone } from '@pinecone-database/pinecone';
import { PineconeStore } from '@langchain/pinecone';
import { Document as LCDocument } from '@langchain/core/documents';

export interface VectorStoreConfig {
  indexName: string;
  namespace?: string;
}
```



```

}

export class VectorStoreManager {
  private embeddings: OpenAIEmbeddings;
  private pinecone: Pinecone;
  private config: VectorStoreConfig;

  constructor(config: VectorStoreConfig) {
    this.embeddings = new OpenAIEmbeddings({ model: 'text-embedding-3-small', dimensions:
1536 });
    this.pinecone = new Pinecone({ apiKey: process.env.PINECONE_API_KEY! });
    this.config = config;
  }

  // 📁 添加文档到向量库
  async addDocuments(docs: Array<{ content: string; metadata?: Record<string, unknown> }>):
Promise<void> {
    const index = this.pinecone.Index(this.config.indexName);
    const lcDocs = docs.map(d => new LCDocument({ pageContent: d.content, metadata:
d.metadata || {} }));
    await PineconeStore.fromDocuments(lcDocs, this.embeddings, { pineconeIndex: index,
namespace: this.config.namespace });
  }

  // 🔍 语义搜索
  async search(query: string, topK = 5): Promise<Array<{ content: string; metadata:
Record<string, unknown>; score: number }>> {
    const index = this.pinecone.Index(this.config.indexName);
    const vectorStore = new PineconeStore(this.embeddings, { pineconeIndex: index,
namespace: this.config.namespace });
    const results = await vectorStore.similaritySearchWithScore(query, topK);
    return results.map(([doc, score]) => ({ content: doc.pageContent, metadata: doc.metadata
as Record<string, unknown>, score }));
  }
}

```

2.8 RAG 链构建

```

// lib/rag-chain.ts
import { ChatOpenAI } from '@langchain/openai';
import { PromptTemplate } from '@langchain/core/prompts';
import { StringOutputParser } from '@langchain/core/output_parsers';
import { RunnableSequence, RunnablePassthrough } from '@langchain/core/runnables';
import { VectorStoreManager } from './vector-store';

export interface RAGResponse { answer: string; sources: Array<{ content: string; score:
number }> }

export class RAGChain {
  private llm: ChatOpenAI;
  private vectorStore: VectorStoreManager;
  private chain: RunnableSequence;

```

```

constructor(vectorStore: VectorStoreManager) {
  this.vectorStore = vectorStore;
  this.llm = new ChatOpenAI({ model: 'gpt-4o', temperature: 0.3 });
  this.chain = this.buildChain();
}

private buildChain(): RunnableSequence {
  const prompt = PromptTemplate.fromTemplate(`
你是一个专业的知识助手。请基于以下参考资料回答用户问题。
如果资料中没有相关信息，请明确说明。

参考资料：
{context}

用户问题: {question}
回答: `);

  const retriever = RunnablePassthrough.fromConfig<{ question: string }>({
    async (input) => {
      const results = await this.vectorStore.search(input.question, 5);
      return results.map(r => r.content).join('\n\n---\n\n');
    }
  });

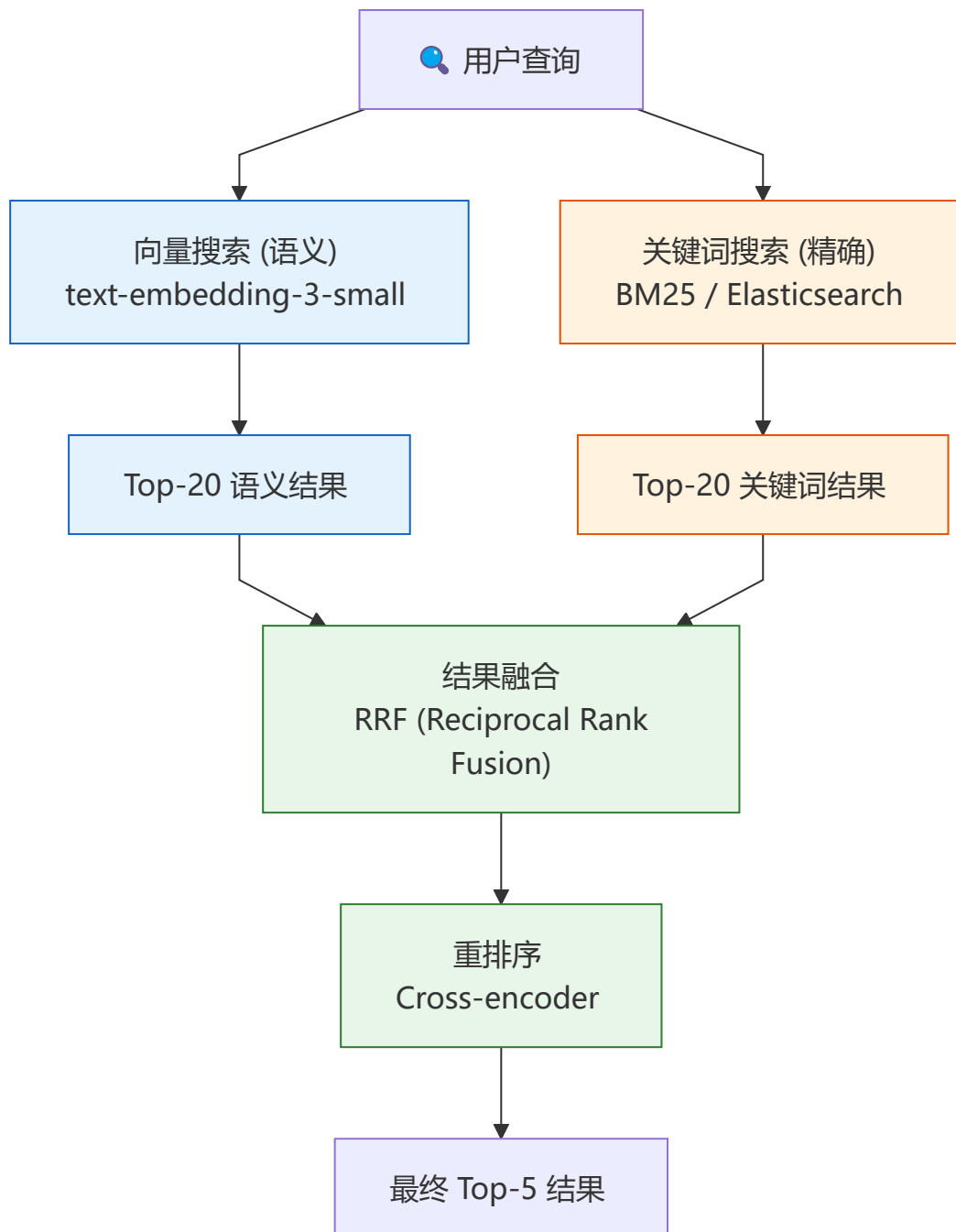
  return RunnableSequence.from([
    { context: retriever, question: new RunnablePassthrough() },
    prompt,
    this.llm,
    new StringOutputParser(),
  ]);
}

async query(question: string): Promise<RAGResponse> {
  const [sources, answer] = await Promise.all([
    this.vectorStore.search(question, 5),
    this.chain.invoke(question),
  ]);
  return { answer, sources };
}

```

混合检索 (Hybrid Search)

向量 + 关键词融合： 向量搜索捕获语义相似性，关键词搜索保证精确匹配，二者互补。



RRF (Reciprocal Rank Fusion) 算法

```
// lib/hybrid-search.ts
export class HybridSearch {
  private vectorStore: VectorStoreManager;

  async search(
    query: string,
    options: {
      topK: number;
      alpha: number; // 0 = 纯关键词, 1 = 纯向量
    } = { topK: 5, alpha: 0.5 }
  ): Promise<ScoredResult[]> {
    // 1. 向量搜索
    const vectorResults = await this.vectorStore.search(query, 20);
```

```

// 2. 关键词搜索 (BM25)
const keywordResults = await this.keywordSearch(query, 20);

// 3. RRF 融合
const fused = this.rrfFusion(vectorResults, keywordResults, 60);

// 4. 按 alpha 加权重排序
const weighted = fused.map(item => ({
  ...item,
  score: options.alpha * item.vectorScore
    + (1 - options.alpha) * item.keywordScore,
}));

return weighted.sort((a, b) => b.score - a.score).slice(0, options.topK);
}

private rrfFusion(
  vector: ScoredResult[],
  keyword: ScoredResult[],
  k: number = 60
): RankedResult[] {
  const rankMap = new Map<string, { vectorRank: number; keywordRank: number }>();

  vector.forEach((item, i) => {
    rankMap.set(item.id, { vectorRank: i + 1, keywordRank: Infinity });
  });
  keyword.forEach((item, i) => {
    const existing = rankMap.get(item.id);
    rankMap.set(item.id, {
      vectorRank: existing?.vectorRank ?? Infinity,
      keywordRank: i + 1,
    });
  });

  return Array.from(rankMap.entries()).map(([id, ranks]) => ({
    id,
    rrfScore: (1 / (k + ranks.vectorRank)) + (1 / (k + ranks.keywordRank)),
    vectorScore: 1 / (k + ranks.vectorRank),
    keywordScore: 1 / (k + ranks.keywordRank),
  })).sort((a, b) => b.rrfScore - a.rrfScore);
}
}

```

检索方式	优势	劣势	适用场景
纯向量搜索	语义理解强，同义词匹配	精确匹配弱，罕见词召回差	开放域问答
纯关键词搜索 (BM25)	精确匹配强，速度快	无法理解语义	专有名词搜索、代码搜索

检索方式	优势	劣势	适用场景
混合检索	兼顾语义 + 精确	多一次检索，延迟增加	生产级 RAG（推荐）

高级分块策略对比

分块质量 = RAG 质量：分块策略直接影响检索准确率，不同文档类型需要不同策略。

策略	实现方式	最佳文档类型	检索准确率	块数控制
固定长度	按字符/Token 数硬切	结构统一的文本	★★	✓
递归分块	按段落 → 句子 → 字符递归切分	通用文档（推荐）	★★★	✓
语义分块	检测主题/语义边界	长文、研究报告	★★★★	✗
Agentic 分块	LLM 判断自然断点	复杂混合内容	★★★★★	✗
特定格式	解析 HTML/Markdown/代码 AST	结构化内容	★★★★	✓

```
// 语义分块实现（基于 Embedding 相似度检测主题转折）
export class SemanticChunker {
  async split(text: string, maxChunkSize = 1000): Promise<string[]> {
    // 1. 按句子分割
    const sentences = text.match(/[\u0300-\u0360\u0390-\u03A0\u03C0-\u03D0\u03E0-\u03F0\u0300-\u0360\u0390-\u03A0\u03C0-\u03D0\u03E0-\u03F0]+[\u0300-\u0360\u0390-\u03A0\u03C0-\u03D0\u03E0-\u03F0]/g) || [text];

    // 2. 对每对相邻句子计算语义相似度
    const embeddings = await this.embedder.embedBatch(sentences);
    const breaks: number[] = [0];

    for (let i = 1; i < sentences.length; i++) {
      const similarity = cosineSimilarity(embeddings[i - 1], embeddings[i]);
      if (similarity < 0.5) breaks.push(i); // 语义转折点
    }

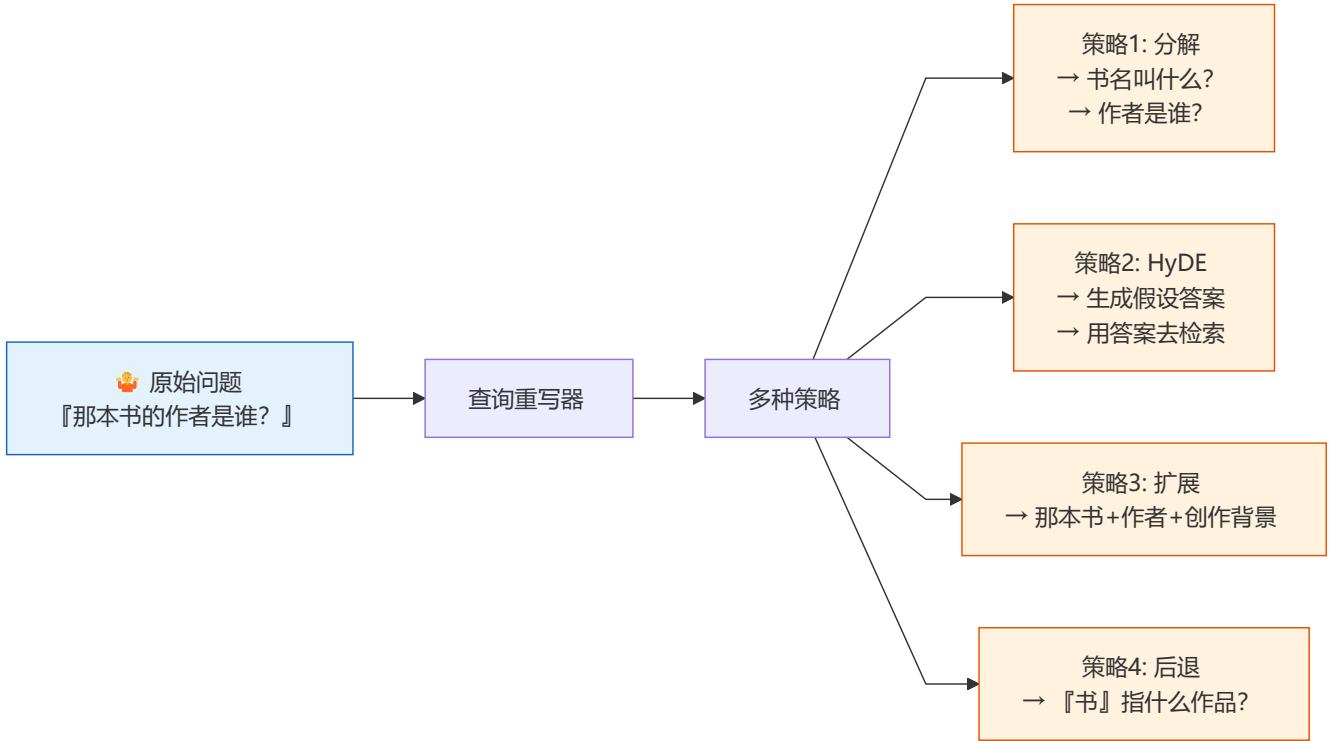
    // 3. 按转折点合并段落，不超过 maxChunkSize
    return this.mergeChunks(sentences, breaks, maxChunkSize);
  }

  private cosineSimilarity(a: number[], b: number[]): number {
    const dot = a.reduce((s, v, i) => s + v * b[i], 0);
    const normA = Math.sqrt(a.reduce((s, v) => s + v * v, 0));
    const normB = Math.sqrt(b.reduce((s, v) => s + v * v, 0));
    return dot / (normA * normB);
  }
}
```



查询重写与扩展 (Query Transformation)

提升检索命中率：用户原始问题往往不适合直接检索，需要"转化"为更易匹配的形式。



```
// 查询重写器
export class QueryRewriter {
  constructor(private llm: ChatOpenAI) {}

  async rewrite(original: string, strategy: 'decompose' | 'hyde' | 'expand' | 'stepback'): Promise<string[]> {
    const prompts: Record<string, string> = {
      decompose: `将以下问题分解为 2-3 个更具体的子问题: \n${original}`,
      hyde: `假设你已经知道了答案, 请生成一段包含详细信息的假设回答: \n${original}`,
      expand: `扩展以下查询, 补充同义词和相关术语: \n${original}`,
      stepback: `为了回答这个问题, 我们需要先知道什么更一般的信息? \n${original}`,
    };

    const response = await this.llm.invoke(prompts[strategy]);
    return this.parseQueries(response.content);
  }
}
```



检索质量评估体系

无评估，不优化：建立量化指标是持续改进 RAG 的前提。

指标	计算方式	目标值	测量方法
命中率 (Hit Rate)	检索结果包含答案的比例	> 85%	人工标注的 Q&A 测试集

指标	计算方式	目标值	测量方法
MRR	首个正确答案的平均倒数排名	> 0.8	同上
NDCG@K	考虑排序位置的累积增益	> 0.75	多级相关性评分
检索延迟 P95	检索阶段 95% 分位延迟	< 500ms	生产环境采样
上下文利用率	最终回答实际引用的检索片段比例	> 60%	LLM 输出解析

```
// 评估脚本
class RAGEvaluator {
  async evaluate(testSet: QAPair[]): Promise<EvaluationReport> {
    let hits = 0;
    const reciprocalRanks: number[] = [];

    for (const { question, groundTruth } of testSet) {
      const results = await this.retriever.search(question, 5);

      // 检查是否有结果包含标准答案
      const hitIndex = results.findIndex(r =>
        this.containsGroundTruth(r.content, groundTruth)
      );

      if (hitIndex >= 0) {
        hits++;
        reciprocalRanks.push(1 / (hitIndex + 1));
      }
    }

    return {
      hitRate: hits / testSet.length,
      mrr: reciprocalRanks.reduce((a, b) => a + b, 0) / testSet.length,
      totalTests: testSet.length,
    };
  }
}
```

🏆 阶段二实战项目

项目	难度	核心考察点	完成标准
🟢 个人知识库	☆☆	文档解析、分块、向量化	支持 PDF/TXT 上传与检索
🟡 智能问答系统	☆☆☆	RAG 链、引用来源、流式输出	回答准确，附带参考链接
🟣 高级检索	☆☆☆☆	混合检索、重排序、查询扩展	检索准确率 (Hit Rate) > 80%

导航

 [上一阶段：入门期](#)

 [学习指南总览](#)

 [下一阶段：深耕期 - 端侧推理](#)